

Mitigating Lost in Multi-turn Conversation via Curriculum RL with Verifiable Accuracy and Abstention Rewards

Ming Li^{1,2*}, Pei Chen², Zhenhao Zhang², Tao Yang², Xinyang Zhang², Han Li², Tianyu Cao², Ming Zeng², Zhuofeng Wu², Meng Jiang², Huasheng Li², Lihong Li², Bing Yin²

¹University of Maryland ²Amazon
minglii@umd.edu ppeichen@amazon.com

Abstract

Large Language Models demonstrate strong capabilities in single-turn instruction following but suffer from Lost-in-Conversation (LiC), a degradation in performance as information is revealed progressively in multi-turn settings. Motivated by the current progress on Reinforcement Learning with Verifiable Rewards (RLVR), we propose Curriculum Reinforcement Learning with Verifiable Accuracy and Abstention Rewards (RLAAR), a framework that encourages models not only to generate correct answers, but also to judge the solvability of questions in the multi-turn conversation setting. Our approach employs a competence-gated curriculum that incrementally increases dialogue difficulty (in terms of instruction shards), stabilizing training while promoting reliability. Using multi-turn, on-policy rollouts and a mixed-reward system, RLAAR teaches models to balance problem-solving with informed abstention, reducing premature answering behaviors that cause LiC. Evaluated on LiC benchmarks, RLAAR significantly mitigates LiC performance decay (62.6% to 75.1%) and improves calibrated abstention rates (33.5% to 73.4%). Together, these results provide a practical recipe for building multi-turn reliable and trustworthy LLMs.

1 Introduction

Large Language Models (LLMs) have demonstrated a remarkable ability to comprehend and execute complex, fully-specified instructions in single-turn settings (Grattafiori et al., 2024; Qwen et al., 2025; Guo et al., 2025; Yang et al., 2025). This proficiency in instruction following has established them as powerful general-purpose problem solvers. However, this success covers a critical weakness that emerges in more natural, interactive scenarios: when instructions are revealed progressively in multi-turn conversations, models get “Lost in

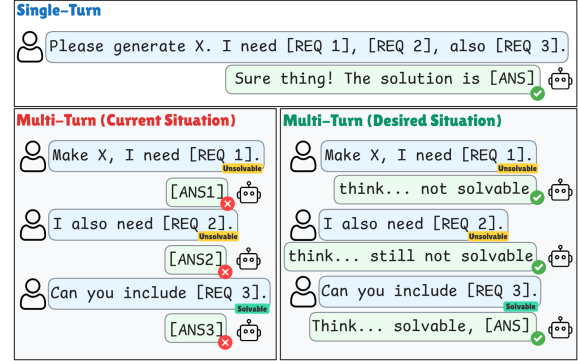


Figure 1: The common single-turn interactive setting with LLMs (upper) and the multi-turn setting proposed in “Lost in Conversation” (LiC). The multi-turn setting simulates the real-world scenario where the user gradually reveals the requirements of the task through multiple turns of dialogue with LLMs. Laban et al. (2025) shows even the state-of-the-art models encounter severe performance declines. The left panel shows the situation of most of the current LLMs, which try to solve the problem prematurely without checking the solvability of the question. The right panel shows the desired behavior of the model, where we novelly reveal the connections between LiC and models’ abstention capability, which critically checks the solvability of the question and then solves the problem.

Conversation” (LiC) (Laban et al., 2025) (Figure 1). This phenomenon reveals a stark degradation in performance in multi-turn dialogues compared to single-turn baselines. The large-scale experiments show that this is not a failure of the models’ underlying aptitude, as their best-case ability remains high, but rather a catastrophic increase in unreliability. In multi-turn settings, models frequently answer the question prematurely and subsequently fail to recover.

The LiC phenomenon stems from a set of identifiable behavioral flaws. Existing models lack the ability to abstain when facing unsolvable questions (Yin et al., 2023; Fan et al., 2025; Wen et al., 2025) and thus tend to generate full solutions prematurely,

*This work was completed during an internship at Amazon.

polluting the context with flawed assumptions and outputs (Figure 1, left). This behavior is exacerbated by failure to identify the crucial details from intermediate turns, extending the well-documented “lost-in-the-middle” (Liu et al., 2024) problem from static long-context inputs to the temporal dimension of a dialogue. Solving this problem is critical, as gradual instruction refinement is the natural mode of interaction for complex, real-world tasks. The failure of LLMs to maintain reliability in these settings limits their adoption, serving as a major barrier for trustworthy AI (Li et al., 2022a).

To overcome these behavioral patterns, a training paradigm is required to allow the model to explore conversational strategies rather than merely mimicking the static trajectories found in a supervised dataset. We therefore turn to Reinforcement Learning (RL) (Schulman et al., 2017; Ouyang et al., 2022; Guo et al., 2025), but with a crucial adaptation for the multi-turn setting. Most existing RL applications treat prior turns as a fixed context for a single-step rollout, which still fails to capture the interactive dynamics of a dialogue. Our approach implements a **fully dynamic, multi-turn rollout** during policy optimization. In each training step, we simulate an entire conversation where the model’s generated response at any given turn becomes part of the state for the subsequent turn. This design is critical: it allows the model to explore on every turn, and to learn from the downstream consequences of its intermediate actions. To ground this complex exploratory process, we use verifiable rewards (RLVR) (Guo et al., 2025), such as correctness determined by a code interpreter or a math solver. These objective, ground-truth signals provide a stable and reliable learning target, which is essential for guiding the policy updates within the vast state space created by our dynamic rollouts.

However, an RL framework that optimizes solely for final task accuracy is itself insufficient. Such a setup implicitly encourages the model to attempt a full solution at every turn, as this is the only path to a potential reward, similar to the findings in Kalai et al. (2025). This reinforces the very behavior of premature answering that causes LiC. The model pollutes the context with its own guesses, making it progressively harder to integrate the user’s subsequent clarifications correctly. To counteract this, we introduce a crucial second component: an abstention reward. This novel reward signal creates an alternative, valuable action for the model: to recognize that the current information is insufficient

and to deliberately output an abstention (Yin et al., 2023; Fan et al., 2025). By explicitly rewarding the model for knowing what it does not know, we train it to shift from a guess-and-check strategy to a more patient, information-gathering policy.

Finally, learning such a nuanced conversational policy, balancing task progression with judicious abstention, presents a significant credit assignment problem. In our multi-turn setting, a natural and effective proxy for this difficulty is the number of conversational turns. Training a model from scratch on long dialogues (e.g., five or more turns) is highly inefficient, as the probability of the model generating a correct, verifiable final response is exceedingly low. This results in a sparse reward landscape, making it difficult for the RL algorithm to learn an effective policy. We therefore introduce curriculum learning (Soviany et al., 2022) to make this problem tractable. Our curriculum begins with short, simple conversations, where the connection between actions and a positive reward is more direct. This allows the model to first master the core skills of context integration and abstention in a high-signal environment. The conversational length and complexity are then progressively increased to medium and difficult scenarios, enabling the model to stably build upon its learned strategies.

With the above discussions, we propose a novel framework, Curriculum Reinforcement Learning with Verifiable Accuracy and Abstention Rewards (RLAAR), to mitigate the LiC problem. By comparing our method with baseline models and state-of-the-art models, we show that our method can significantly improve the performance and help the model to judge the solvability of the question in LiC settings.

The primary contributions of this work are:

1. We propose a novel reinforcement learning framework that optimizes a model’s conversational policy to mitigate LiC, moving beyond static imitation learning to foster more robust, exploratory decision-making.
2. We **novelly reveal the connection between LiC and models’ abstention capability** and design a **verifiable, mixed-reward system** that uniquely combines accuracy rewards for task completion with abstention rewards for handling ambiguity.
3. We demonstrate that a **curriculum learning strategy** is critical for training stability, enabling the model to learn complex multi-turn

behaviors by progressively increasing conversational depth and difficulty.

2 Related Works

2.1 Multi-turn Conversation

As interest in multi-turn evaluation surged alongside the meteoric rise of ChatGPT, early widely used efforts like MT-bench (Zheng et al., 2023) relied on crowd-sourced annotations to assess LLM-as-a-judge performance. Subsequent work extended MT-bench by supporting longer dialogues (Kwan et al., 2024; Duan et al., 2024), offering finer-grained assessment (Bai et al., 2024), and targeting new axes such as naturalness (Sirdeshmukh et al., 2025) or tool use (Yan et al., 2024; Wang et al., 2024). While these tasks require some context tracking, they rarely demand the active integration needed to resolve underspecified user instructions. Such underspecification is both common in real human-AI exchanges (Herlihy et al., 2024) and a natural conversational tendency captured by the “principle of least effort” (Zipf, 1949). Scaling multi-turn experimentation, therefore, requires simulating the user. Prior studies have done so via templates (Choi et al., 2018; Reddy et al., 2019; Laban et al., 2023; Deng et al., 2024a), LLMs (Poelitz and McKenna, 2025; Li et al., 2024; Chang et al., 2025; Liang et al., 2024), human annotators (Finch et al., 2023; Chang et al., 2025), or real users in controlled studies (Ram et al., 2018; Laban et al., 2021; Chiang et al., 2024). While real users yield the most natural interactions, this approach sacrifices scalability and reproducibility. LiC therefore adopts an LLM-based simulator to balance control with diversity, enabling the study of *LLM behavior* in multi-turn contexts and exposing the core issues underlying the LiC phenomenon, the central focus of this work.

2.2 Response Abstention

One key capability for mitigating the Lost in Conversation problem is an LLM’s ability to judge a query’s solvability and abstain from unsolvable ones, so the model can, to some extent, avoid issuing premature, verbose answers that further pollute the context. Numerous datasets evaluate abstention, but they typically target a single problem type, e.g., unanswerable questions (Yin et al., 2023; Amayuelas et al., 2024), multiple-choice questions with no correct option (Madhusudhan et al., 2025), or underspecification (Slobodkin

et al., 2023; Zhang et al., 2024; Li et al., 2025). Closely related is verbalized uncertainty (Lin et al., 2022; Tian et al., 2023), the model’s explicit expression of doubt as a downstream signal that it cannot appropriately answer. Several works report limited performance and generalization for verbalized uncertainty as an uncertainty-quantification method (Vashurin et al., 2024; Lin et al., 2022; Xiong et al., 2024). At the same time, Kapoor et al. (2024) shows that fine-tuning can improve verbalized uncertainty, and Kadavath et al. (2022) demonstrates that suitable prompting can elicit correctness probabilities that become increasingly calibrated as models scale. Parallel efforts have improved abstention via fine-tuning (Chen et al., 2024; Brahman et al., 2024) and through explanation generation (Deng et al., 2024b). Prior work has also benchmarked and aimed to improve policy compliance (Brahman et al., 2024; Mueller et al., 2024; Mazeika et al., 2024). Recently, as the growing interest in Large Reasoning Models, MiP-Overthinking (Fan et al., 2025) further reveals an interesting phenomenon that a longer thinking process not only does not help the model to abstain from the unsolvable question, but also exacerbates the problem. In our work, our method has a better abstention ability in this LiC setting.

2.3 RL in LLMs

Reinforcement Learning has become a powerful paradigm for aligning LLMs with human preferences and values. The most widely used method is Proximal Policy Optimization (PPO) (Ouyang et al., 2022; Schulman et al., 2017). Building on PPO, GRPO removes the need for an explicit value function by estimating advantages from group scores and has become a popular training paradigm (Shao et al., 2024). Reinforcement Learning with Verifiable Rewards (RLVR) further streamlines training by evaluating outputs with predefined rules, eliminating the reward model (Guo et al., 2025). Variants that address specific limitations of GRPO have also been proposed (Su et al., 2025). To mitigate length bias, Dr. GRPO drops normalization terms, while SRPO adds two-stage training with historical sampling (Liu et al., 2025; Zhang et al., 2025). To improve training stability, DAPO (Yu et al.) increases clip rates, and GVPO (Zhang et al., 2025) provides an analytical solution to KL-constrained reward maximization. Compared with prior work, our method utilizes multi-turn rollouts and a mixed reward system to train the model to

mitigate the LiC problem.

3 Methodology

3.1 Problem Formulation

We cast multi-turn, incrementally specified dialog tasks defined in Laban et al. (2025) as a finite-horizon Markov Decision Process with a verifiable terminal signal. A complete task T is represented as an ordered sharded question sequence $\mathcal{S} = \{s_1, \dots, s_K\}$, where each shard question s_k is a subset of the original complete question q and encodes the user’s newly revealed constraints at round k . At turn k , the policy π_θ consumes the context $c_k = (s_1, a_1, \dots, s_k)$ and emits a textual action a_k . At the final round K , the model returns a terminal answer a_K . The environment provides a single terminal reward, either from an accuracy verifier (solvable cases) or from an abstention verifier (unsolvable cases).

For a trajectory $\tau = (c_1, a_1, \dots, c_K, a_K)$, we define

$$r(\tau) = \begin{cases} r_{\text{acc}}(a_K; T) \in \{0, 1\}, & \text{if } T \text{ is solvable,} \\ r_{\text{abs}}(a_K; T) \in \{0, 1\}, & \text{if } T \text{ is unsolvable.} \end{cases} \quad (1)$$

$r_{\text{acc}}(a_K; T)$ represents the verifiable accuracy reward and $r_{\text{abs}}(a_K; T)$ represents the verifiable abstention reward for the terminal answer a_K on the task T . The overall pseudo code of the RLAAR method is provided in Appendix C.

3.2 Curriculum RL with Verifiable Accuracy and Abstention Rewards

3.2.1 Multi-turn Rollout

Different from most of the existing RL methods for LLMs that only focus on the single-turn setting, our method novelly utilizes on-policy multi-turn rollouts that explicitly simulate the dialogue process. Specifically, there are three different types of rollouts during the training:

- *Solvable-Single*: the model is given the original full question and expected to generate the final answer in one action.
- *Solvable-Multi*: the model is given one of the sharded questions each turn until all shards are revealed and expected to generate the final answer after multiple actions.
- *Unsolvable-Multi*: the model is given one of the sharded questions each turn, while some shards are intentionally withheld and expected to *abstain* from the unsolvable question.

Solvable-Single. In this setting, the episode consists of a single turn ($K = 1$). The complete task description q is given as the first and only shard, $s_1 = q$. The policy generates a single action $a_1 \sim \pi_\theta(a|s_1)$. The resulting trajectory is $\tau = (s_1, a_1)$, and the policy is optimized to maximize $\mathbb{E}[r_{\text{acc}}(a_1; T)]$. This setting is equivalent to the single-turn setting in the existing RL methods for LLMs. It is only utilized at the beginning of the training.

Solvable-Multi. In this setting, the full sequence of K shards, $\mathcal{S} = \{s_1, \dots, s_K\}$, is revealed to the model sequentially. At the beginning of each turn $k \in \{1, \dots, K\}$, the environment presents the next shard s_k . The policy then conditions on the accumulated context $c_k = (s_1, a_1, \dots, s_{k-1}, s_k)$ to sample an action $a_k \sim \pi_\theta(a|c_k)$. This process repeats until the full trajectory $\tau = (s_1, a_1, \dots, s_K, a_K)$ is generated. The optimization objective is to maximize the expected final reward $\mathbb{E}[r_{\text{acc}}(a_K; T)]$. This setting is directly aligned with the evaluation setting presented in LiC, and thus serves as the main training setting for the model.

Unsolvable-Multi. This setting is similar to the *Solvable-Multi* case, but the interaction is truncated after $M < K$ turns. The environment provides shards $s_1, \dots, s_M$ sequentially, and the policy generates corresponding actions $a_1, \dots, a_M$. After the environment presents the final shard s_M and the policy produces action a_M , the episode terminates. The resulting trajectory is $\tau = (s_1, a_1, \dots, s_M, a_M)$, and the policy is trained to produce a terminal action a_M that explicitly abstains, maximizing the expected reward $\mathbb{E}[r_{\text{abs}}(a_M; T)]$. Different from the above settings, which focus on the model’s correctness, this setting is designed to focus on the model’s capability to justify the solvability of the question. It also serves as the main training setting.

In the main training process, we further introduce the abstention ratio $m \in [0, 1]$ to control the proportion of the *Unsolvable-Multi* rollouts. When $m = 0$, the model is only trained on the *Solvable-Multi* rollouts. When $m = 1$, the model is only trained on the *Unsolvable-Multi* rollouts. This hyperparameter is critical for the model to learn the balance between correctness and abstention. Note that for both the multi-turn settings, the rollout process is fully *dynamic and on-policy*, rather than just using the static trajectory as the context. Thus, the

model can explore on every turn and learn from the downstream consequences of its intermediate actions.

3.2.2 Mixed Rewards

A standard RLVR (Guo et al., 2025) setup that optimizes solely for question accuracy is insufficient for the LiC problem. In fact, such a reward scheme would implicitly encourage the model to attempt a full solution at every turn, as this is the only path to a potential reward, rather than to first evaluate the solvability of the current context. This reinforces the very behavior of premature answering we aim to eliminate. To address this, we introduce a mixed-reward system that provides distinct, verifiable signals for two contrary but equally desirable behaviors: question accuracy and strategic abstention. As defined in Equation 3.1, our reward function relies on two verifier functions, V_{acc} and V_{abs} , which formalize the reward calculation.

Accuracy Reward (r_{acc}). This reward is the standard accuracy reward used for existing RLVR methods, which rewards the model for correctly solving the question. In our setting, it is applied to both the *Solvable-Single* and *Solvable-Multi* settings. Specifically, we define an automatic accuracy verifier, $V_{\text{acc}}(a_K, T)$, which returns ‘true’ if the model’s final answer a_K correctly solves the complete task T , and ‘false’ otherwise. The reward is then formally defined with an indicator $\mathbb{I}(\cdot)$:

$$r_{\text{acc}}(a_K; T) = \mathbb{I}(V_{\text{acc}}(a_K, T) = \text{true}) \quad (2)$$

This assigns a reward of 1 for a correct answer and 0 otherwise. Note that for different types of tasks, the verifier V_{acc} is different: For the math tasks, the verifier checks if the model’s answer is the same as the ground truth answer and wrapped in the predefined format, i.e., “[x]”. For the code tasks, the verifier checks if the model’s generated code is executable and can successfully pass the test cases.

Abstention Reward (r_{abs}). This reward is designed for the *Unsolvable-Multi* setting, aiming to train the model to abstain from the unsolvable question. The reward is determined by a simple verifier, $V_{\text{abs}}(a_M)$, which checks if the model’s terminal action a_M contains the predefined string representing the abstention behavior, i.e., “[*Abstain*]”. The reward is task-agnostic and only depends on the model’s terminal action. The reward is then:

$$r_{\text{abs}}(a_M; T) = \mathbb{I}(V_{\text{abs}}(a_M) = \text{true}) \quad (3)$$

This assigns a reward of 1 for correctly abstaining and 0 for any other action, such as attempting to provide a premature answer. Since only string-matching is required for this reward, the calculation is efficient.

The combination of these two reward signals is crucial. It creates a balanced incentive structure that teaches the model a critical meta-skill: to first evaluate the solvability of the current context before committing to an answer. By providing a valuable alternative to guessing (i.e., abstaining for a reward), the model learns to handle the ambiguity inherent in multi-turn, incrementally specified dialogues, which is the key to mitigating the LiC phenomenon, which is important to trustworthy AI.

3.2.3 Curriculum Design

Training an RL policy on long multi-turn dialogues from scratch is naturally difficult. The probability of generating a correct final response after a long sequence of intermediate actions is exceedingly low, leading to a sparse reward landscape that makes the learning process unstable and inefficient. To address this challenge, we introduce a curriculum learning strategy that makes the complex learning problem tractable by gradually increasing the conversational difficulty.

Our curriculum structures the learning process by naturally defining difficulty based on the number of shards K the original complete question is segmented into. A higher K corresponds to a longer and more challenging conversation, which is naturally more difficult to solve. The curriculum unfolds in three main stages.

Stage 1: Threshold Establishment. The training process begins exclusively with the *Solvable-Single* setting. The model is trained on original, complete problems where $K = 1$. After an initial training period, we calculate the moving-average reward $\bar{r}_0$ over a predefined window (5 by default). This value serves as a dynamic, model-specific performance baseline that reflects the model’s initial performance on the easiest problems. With an additional threshold ratio $\rho \in [0, 1]$, we use $\rho \times \bar{r}_0$ as the real threshold for the subsequent stages.

Stage 2: Main Training. After establishing the baseline reward, the curriculum introduces multi-turn tasks, *Solvable-Multi* and *Unsolvable-Multi*, starting with $K = 2$ for each training batch. The model continues to train at a given difficulty level K until its current moving-average reward over a

window surpasses the predefined threshold. Formally, the curriculum progresses from K shards to $K + 1$ shards only when the condition $\bar{r} \geq \rho \times \bar{r}_0$ is met. This process repeats until a predefined maximum shard count, $K_{\max}$, is reached. The reason that an additional threshold ratio ρ is introduced is that the model’s performance on harder scenarios is naturally lower than in the single-turn setting. Thus, if no such scaling is applied, the model may get stuck and never progress to the next difficulty level.

Stage 3: Randomized Training. Once the model has successfully passed the maximum difficulty level, the curriculum transitions to its final stage. In this phase, for each training batch, the number of shards K is randomly sampled between 1 and $K_{\max}$. This randomization encourages a more robust and generalized policy that can handle conversations of varying lengths, rather than overfitting to the maximum difficulty level.

4 Experiments

4.1 Implementation Details

Models. We implement our method on Qwen3-8B (Yang et al., 2025), Qwen3-1.7B, Qwen2.5-7B-Instruct (Qwen et al., 2025). All the models are instruction-aligned versions. During training, we use the default chat template for each model. For the Qwen3 model families, we utilize the non-reasoning version of the conversation template for both training and testing.

Data. Our data mainly includes two subcategories: the math dataset and the code dataset. For the Math dataset, we utilize the GSM8K (Cobbe et al., 2021) dataset as our source dataset. For the Code dataset, we utilize the Euris (Cui et al., 2025) code subset, including code generation problems sourced from TACO (Li et al., 2023), Codecontests (Li et al., 2022b), Codeforces, and apps (Hendrycks et al., 2021) datasets. For each math or codesubcategory, we randomly sample 1000 problems that are long enough to be segmented into multiple sharded instructions for training.

Training Details. Our code is implemented based on the Ver1¹ framework, and we conduct all the experiments on 8 A100 GPUs. We use the AdamW optimizer with a learning rate of $1e-4$ and a weight decay of 0.1. We use a batch size of 64

for training. The experiments are conducted on the GRPO (Shao et al., 2024) algorithm by default. In GRPO, the group size is set to 4, and we use the default settings for the other hyperparameters. The training utilizes a dynamic model-specific stopping strategy, where the maximum training step is set to 100 steps after the model gets into the Randomized Training stage (stage 3). For method-specific parameters, the default maximum shard number is $K_{\max} = 5$; the default abstention ratio is $m = 0.1$; the default threshold ratio is $\rho = 0.8$.

Evaluation. To evaluate the performance of our method, we focus on *LiC Score* and the *Abstain Score*. The *LiC Score* is the ratio of the problems that can be correctly solved in the concatenated instruction setting but not in the sharded instruction setting, which is calculated as $\frac{Accuracy(Sharded)}{Accuracy(Concatenated)}$. This is the standard metric used in LiC and presented as the main findings that even the recent state-of-the-art models have the LiC score of around 0.6. However, as mentioned in the previous section, this metric actually encourages the model to generate answers each turn prematurely, regardless of the solvability of the current context, which is not desired behavior. Therefore, we also introduce the *Abstain Score*, the ratio of the problems that are correctly abstained in the sharded instruction setting, which is calculated as $\frac{Abstention(UnsolvableSharded)}{Total(UnsolvableSharded)}$. For this metric, we utilize the LLM-as-a-judge strategy that prompts GPT4o to judge whether the model’s answer mentioning the current question is not solvable yet. This metric is more aligned with the goal of trustworthy AI and is more interpretable for humans to understand the model’s behavior. Note, although during training, we encourage the model to generate “[*Abstain*” for calculating rewards, we avoid utilizing the simple string matching strategy for evaluation for fair comparison with other models. Utilizing the LLM-as-a-judge, we can better evaluate the model’s response if it expresses the abstain behavior but does not explicitly use the predefined string.

4.2 Main Results

Table 1 shows the performance comparison of previous well-known models. Performance on 4 different tasks is presented, including math, code, database, and actions; *Concat* and *Sharded* represent the performance of the concatenated instruction setting and the sharded instruction setting, re-

¹<https://github.com/volcengine/ver1>

Model	Math				Code				Database				Actions				Average	
	Concat	Sharded	LiC	Abstain	C	S	L	A	C	S	L	A	C	S	L	A	LiC	Abstain
OLMo2 (OLMo et al., 2024)	80.1	46.3	57.8	-	16.3	14.4	88.3	-	40.5	22.4	55.3	-	49.8	13.8	27.7	-	57.3	-
Claude3-Haiku (Anthropic, 2024)	76.1	47.1	61.9	-	36.3	31.5	86.8	-	76.5	31.8	41.6	-	80.2	55.9	69.7	-	65.0	-
GPT4o-mini (OpenAI, 2024)	88.0	58.7	66.7	-	66.7	50.3	75.4	-	90.7	40.2	44.3	-	92.2	52.4	56.8	-	60.8	-
Llama3.3-70B (Grattafiori et al., 2024)	91.8	61.5	67.0	-	52.7	51.6	97.9	-	87.9	35.4	40.3	-	97.0	71.0	73.2	-	69.6	-
Phi-4 (Abdin et al., 2024)	90.4	52.5	58.1	-	48.4	39.1	80.8	-	79.6	33.1	41.6	-	76.0	34.1	44.9	-	56.4	-
Llama4-Scout (AI, 2025)	92.9	67.0	72.1	-	60.3	46.4	76.9	-	81.5	27.1	33.3	-	98.3	69.9	71.1	-	63.4	-
GPT-o3 (OpenAI, 2025b)	80.0	63.1	78.9	-	87.2	53.0	60.8	-	83.3	35.4	42.5	-	91.5	60.2	65.8	-	62.0	-
Claude3.7-Sonnet (Anthropic, 2024)	87.2	70.0	80.3	-	76.2	65.6	86.1	-	81.5	34.9	42.8	-	96.0	33.3	34.7	-	61.0	-
DeepSeek-R1 (Guo et al., 2025)	92.9	67.3	72.4	-	97.1	70.9	73.0	-	89.9	31.5	35.0	-	97.0	47.5	49.0	-	57.4	-
GPT-4o (Hurst et al., 2024)	91.9	67.9	73.9	39.9	82.9	61.3	73.9	29.8	91.7	42.3	46.1	29.4	97.1	65.0	66.9	26.8	65.2	31.5
Gemini-2.5-Flash (Comanici et al., 2025)	88.4	66.1	74.8	47.0	92.5	68.3	73.8	26.8	95.5	51.3	53.7	32.8	89.2	42.6	47.8	25.4	62.5	33.0
GPT-4.1 (OpenAI, 2025a)	89.7	70.7	78.8	40.5	88.7	72.6	81.8	27.4	86.5	46.0	53.2	35.8	98.5	62.9	63.9	30.1	69.4	33.5
Gemini-2.5-Pro (Comanici et al., 2025)	89.3	64.3	72.0	46.2	95.7	68.1	71.2	30.2	94.9	43.8	46.2	33.1	98.1	36.3	37.0	32.9	56.6	35.6
Qwen3-1.7B (Grattafiori et al., 2024)	67.6	44.4	65.7	26.8	33.4	15.0	44.9	19.8	51.1	38.8	76.0	20.4	83.1	40.2	48.4	24.9	58.8	23.0
Qwen3-1.7B (Ours, RLAAR)	70.9	60.0	<u>84.7</u>	<u>77.2</u>	37.0	22.9	<u>61.94</u>	<u>69.3</u>	47.7	41.1	<u>92.5</u>	<u>62.0</u>	84.8	41.3	<u>48.7</u>	<u>65.2</u>	71.9	68.4
Qwen3-8B (Grattafiori et al., 2024)	88.4	72.8	82.4	39.1	60.0	35.0	58.3	28.9	76.0	46.1	60.6	31.1	99.0	48.5	49.0	34.7	62.6	33.5
Qwen3-8B (SFT, Distill Gemini-2.5-Pro)	87.4	75.6	86.5	57.4	58.4	34.2	58.6	42.9	67.6	40.2	59.5	47.8	90.2	41.6	46.1	47.8	62.7	49.0
Qwen3-8B (Ours, RLAAR)	87.2	82.2	<u>94.3</u>	<u>80.8</u>	63.2	44.8	<u>70.9</u>	<u>72.3</u>	77.6	63.2	<u>81.4</u>	<u>69.7</u>	97.4	52.3	<u>53.7</u>	<u>69.7</u>	<u>75.1</u>	73.4
Qwen2.5-7B (Qwen et al., 2025)	82.2	59.8	72.8	36.1	55.0	31.0	56.4	26.2	73.2	38.5	52.5	27.7	95.8	35.0	36.5	31.9	54.6	30.5
Qwen2.5-7B (SFT, Distill Gemini-2.5-Pro)	82.7	60.3	72.9	57.2	56.4	33.9	60.1	47.7	69.6	36.4	52.3	42.9	91.7	31.6	34.5	52.7	55.0	50.1
Qwen2.5-7B (Ours, RLAAR)	83.4	79.2	<u>95.0</u>	<u>77.0</u>	58.7	41.2	<u>70.2</u>	<u>72.0</u>	74.9	65.4	<u>87.3</u>	<u>61.3</u>	92.8	49.3	<u>53.0</u>	<u>70.3</u>	76.4	67.6

Table 1: Performance comparison of previous well-known models. From OLMo2 to Gemini-2.5-Pro, we directly borrow the results from the original LiC paper (Laban et al., 2025), except for the abstain scores. *Concat* and *Sharded* represent the performance of the concatenated instruction setting and the sharded instruction setting, respectively; *LiC* and *Abstain* represent the LiC score and the abstain score, respectively. The better performances compared with the baseline models are underlined, and the best average performances across all models are highlighted in bold.

spectively; *LiC* and *Abstain* represent the LiC score and the Abstain score, respectively. The better performance compared with the baseline models is underlined, and the best performance across all models is highlighted in bold.

As shown in the table, even the recent state-of-the-art models like GPT-4.1 and Gemini-2.5-Pro have the average LiC score of around 0.6, which means their performance can only maintain 60% of the performance of the concatenated instruction setting, which shows the severity of the “Lost in Conversation” problem. Moreover, the performance comparison shows that supervised finetuning on relatively strong models like Gemini-2.5-Pro is not effective for mitigating the LiC problem. On the other hand, models trained with our method outperform the baseline models on all tasks to a large margin, represented by the underlined results. Even when compared with the well-known state-of-the-art models, our method can reach the average LiC score of around 0.75, and for math tasks, we can even reach the LiC score of around 0.9, which shows the effectiveness of our method, highlighted by the bold results.

Moreover, our method not only focuses on the LiC score, but also tries to improve the capability of the models to justify the solvability of the current context, thus bringing improvement to the trustworthy AI, which is reflected by the improvement of the abstain score. While most of the models only achieve the abstain score of around 0.2 – 0.4, indicating that they tend to answer questions re-

Abstain Ratio	LiC score	Abstain score
Qwen3-1.7B (Baseline)	58.8	23.0
Qwen3-1.7B ($m = 0$, No Abstention)	64.3	17.4
Qwen3-1.7B ($m = 0.1$, Default)	71.9	68.4
Qwen3-1.7B ($m = 0.2$)	70.8	69.5
Qwen3-1.7B ($m = 0.3$)	62.6	72.8
Qwen3-1.7B ($m = 0.4$)	47.1	80.3
Qwen3-1.7B ($m = 0.5$)	27.4	92.8

Table 2: Ablation studies on the effect of the abstention reward. The baseline model is Qwen3-1.7B, and the abstention ratio m is varied from 0 to 0.5, all other settings are the same as the default setting. The abstention ratio m represents the ratio of the rollouts in each step that are categorized as *Unsolvable-Multi*, thus using the abstention reward rather than the accuracy reward. The best performances are highlighted in bold.

gardless of the solvability, our method can achieve the abstain score of around 0.7 – 0.8, indicating that the model has the capability to justify the solvability of the current context. For more qualitative details, failure and success examples are provided in Appendix B.

4.3 Ablation Studies

4.3.1 Effect of the Abstention Reward

In this section, we conduct ablation studies to investigate the effect of the abstention reward on the performance of the model. As shown in Table 2, the baseline model is Qwen3-1.7B, and the abstention ratio m is varied from 0 to 0.5; all other settings are the same as the default setting. The abstention ratio m represents the ratio of the rollouts in each step that are categorized as *Unsolvable-Multi*, thus using the abstention

Curriculum Setting	LiC score	Abstain score	Steps
Qwen3-1.7B (Baseline)	58.8	23.0	
Qwen3-1.7B (No Curriculum)	63.2	57.8	0
Qwen3-1.7B ($\rho = 1.0$)	65.6	64.3	1000+
Qwen3-1.7B ($\rho = 0.9$)	72.2	66.4	470
Qwen3-1.7B ($\rho = 0.8$, Default)	71.9	68.4	90
Qwen3-1.7B ($\rho = 0.7$)	62.1	60.7	70
Qwen3-1.7B ($\rho = 0.6$)	40.9	47.6	40

Table 3: Ablation studies on the effect of the curriculum learning strategy. The baseline model is Qwen3-1.7B, and the threshold ratio ρ is varied from 0 to 0.5, all other settings are the same as the default setting. The threshold ratio ρ represents the ratio of the baseline threshold that the model’s reward should exceed to progress to the next difficulty level. The best performances are highlighted in bold. In addition to the performance, we also report the steps required to reach the Randomized Training stage (Stage 3), to provide a better understanding of the curriculum learning strategy.

reward rather than the accuracy reward.

When $m = 0$, the model is only trained on the *Solvable-Multi* rollouts using the accuracy reward. Compared with the baseline model, when $m = 0.5$, the model can already achieve the abstain score of 64.3%, which is benefited from the explicit usage of multi-turn rollouts, which forces the model to get the necessary information across previous multi-turn context to solve the problem. However, in this strategy, the model tends to try answering the question in every turn, which causes the growth and potential pollution of the previous context. Thus, the performance is not as good as the default setting.

Thus, when $m > 0$, the models are trained on both accuracy reward and abstention reward, which represents our main method, RLAAR. Through the performance comparison from $m = 0.1$ to $m = 0.5$, we can further observe that the LiC score is gradually decreased while the abstain score is gradually increased. It happens because the abstain strategy is much easier for LLMs to learn; thus, when the ratio of the *Unsolvable-Multi* rollouts is too large, the model tends to overfit the abstain strategy and thus abstain from every request without really solving the problem. In this case, the high abstention score is not useful.

4.3.2 Effect of the Curriculum Learning

In this section, we conduct ablation studies to investigate the effect of the curriculum learning strategy. As shown in Table 3, the baseline model is Qwen3-1.7B, and the threshold ratio ρ is varied from 0 to 0.5; all other settings are the same as the default setting. The threshold ratio ρ represents the ratio of the baseline threshold that the model’s

reward should exceed to progress to the next difficulty level. A higher ρ means the model needs to reach a higher average reward to progress to the next difficulty level.

When no curriculum is applied, the model is trained directly as in Randomized Training stage (Stage 3), where a random number of shards K is sampled between 1 and $K_{\max}$ for each step. As shown in the table, the model has already reached promising performance on both LiC score and abstain score. However, the training process is not as stable as when using the curriculum learning strategy, as it is hard for the model to get positive feedback in the early stage. On the contrary, curriculum learning is naturally suitable for this shard instruction setting, as we can gradually increase the number of shards/turns for the model to train on. However, when $\rho = 1$, which means the model needs to reach an average reward equal to training on a single-turn setting to progress to the next difficulty level, the model gets stuck in every difficulty level, as it’s very hard for the model to reach the single-turn setting performance. As a result, the model needs more than 1000 steps to overfit to the training data and thus pass all the difficulty levels. When ρ is lower than 1, it means the model only needs to reach the average reward of ρ times the baseline threshold to progress to the next difficulty level, which is easier. As a result, the model can progress to the next difficulty level in a more stable way. Comparing $\rho = 0.9$ and $\rho = 0.8$, we can observe that the model performance is similar, but the model needs many fewer steps to pass all the difficulty levels when $\rho = 0.8$, thus we set $\rho = 0.8$ as the default setting. On the other hand, when ρ is too small, the model may pass all the difficulty levels too fast to really learn things.

5 Conclusion

We introduced Curriculum Reinforcement Learning with Verifiable Accuracy and Abstention Rewards (RLAAR) to mitigate the Lost-in-Conversation (LiC) phenomenon in multi-turn language model interactions. By combining verifiable accuracy and abstention rewards within a competence-gated curriculum, our framework enables models to learn when to act and when to defer, leading to more stable and trustworthy dialogue behavior. Experiments demonstrate that RLAAR consistently reduces LiC degradation, enhances reliability through calibrated abstention.

Limitation

While our method demonstrates significant improvements in mitigating Lost-in-Conversation (LiC), our experiments are conducted primarily on math and code datasets, which may not fully capture the complexity of open-domain or knowledge-intensive dialogues. Extending RLAAR to broader conversational settings, such as factual QA or multi-agent collaboration, remains a future direction.

References

- Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, and 1 others. 2024. Phi-4 technical report. [arXiv preprint arXiv:2412.08905](https://arxiv.org/abs/2412.08905).
- Meta AI. 2025. Llama 4: Multimodal intelligence. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- Alfonso Amayuelas, Kyle Wong, Liangming Pan, Wenhui Chen, and William Wang. 2024. Knowledge of Knowledge: Exploring Known-Unknowns Uncertainty with Large Language Models. (arXiv:2305.13712).
- Anthropic. 2024. Claude 3 model card. <https://www.anthropic.com/news/claude-3-family>.
- Ge Bai, Jie Liu, Xingyuan Bu, Yancheng He, Jiaheng Liu, Zhanhui Zhou, Zhuoran Lin, Wenbo Su, Tiezheng Ge, Bo Zheng, and 1 others. 2024. Mt-bench-101: A fine-grained benchmark for evaluating large language models in multi-turn dialogues. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7421–7454.
- Faeze Brahman, Sachin Kumar, Vidhisha Balachandran, Pradeep Dasigi, Valentina Pyatkin, Abhilasha Ravichander, Sarah Wiegrefe, Nouha Dziri, Khyathi Chandu, Jack Hessel, and 1 others. 2024. The art of saying no: Contextual noncompliance in language models. *Advances in Neural Information Processing Systems*, 37:49706–49748.
- Serina Chang, Ashton Anderson, and Jake M Hofman. 2025. Chatbench: From static benchmarks to human-ai evaluation. [arXiv preprint arXiv:2504.07114](https://arxiv.org/abs/2504.07114).
- Lida Chen, Zujie Liang, Xintao Wang, Jiaqing Liang, Yanghua Xiao, Feng Wei, Jinglei Chen, Zhenghong Hao, Bing Han, and Wei Wang. 2024. Teaching Large Language Models to Express Knowledge Boundary from Their Own Signals. (arXiv:2406.10881).
- Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael Jordan, Joseph E Gonzalez, and 1 others. 2024. Chatbot arena: An open platform for evaluating llms by human preference. In *Forty-first International Conference on Machine Learning*.
- Eunsol Choi, He He, Mohit Iyyer, Mark Yatskar, Wentaoh Yih, Yejin Choi, Percy Liang, and Luke Zettlemoyer. 2018. Quac: Question answering in context. [arXiv preprint arXiv:1808.07036](https://arxiv.org/abs/1808.07036).
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. [Training verifiers to solve math word problems](https://arxiv.org/abs/2110.14168). Preprint, arXiv:2110.14168.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, and 1 others. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. [arXiv preprint arXiv:2507.06261](https://arxiv.org/abs/2507.06261).
- Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, and 6 others. 2025. [Process reinforcement through implicit rewards](https://arxiv.org/abs/2502.01456). Preprint, arXiv:2502.01456.
- Yang Deng, Xuan Zhang, Wenxuan Zhang, Yifei Yuan, See-Kiong Ng, and Tat-Seng Chua. 2024a. On the multi-turn instruction following for conversational web agents. [arXiv preprint arXiv:2402.15057](https://arxiv.org/abs/2402.15057).
- Yang Deng, Yong Zhao, Moxin Li, See-Kiong Ng, and Tat-Seng Chua. 2024b. [Don’t Just Say “I don’t know”! Self-aligning Large Language Models for Responding to Unknown Questions with Explanations](https://arxiv.org/abs/2402.15057). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 13652–13673. Association for Computational Linguistics.
- Haodong Duan, Jueqi Wei, Chonghua Wang, Hongwei Liu, Yixiao Fang, Songyang Zhang, Dahua Lin, and Kai Chen. 2024. Botchat: Evaluating llms’ capabilities of having multi-turn dialogues. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3184–3200.
- Chenrui Fan, Ming Li, Lichao Sun, and Tianyi Zhou. 2025. [Missing premise exacerbates overthinking: Are reasoning models losing critical thinking skill?](https://arxiv.org/abs/2502.15057) In *Second Conference on Language Modeling*.
- Sarah E Finch, James D Finch, and Jinho D Choi. 2023. Don’t forget your abc’s: Evaluating the state-of-the-art in chat-oriented dialogue systems. In *The 61st Annual Meeting Of The Association For Computational Linguistics*.

- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and 21 others. 2024. [The llama 3 herd of models](#). [Preprint](#), arXiv:2407.21783.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shitong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. [arXiv preprint arXiv:2501.12948](#).
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and 1 others. 2021. Measuring coding challenge competence with apps. [arXiv preprint arXiv:2105.09938](#).
- Christine Herlihy, Jennifer Neville, Tobias Schnabel, and Adith Swaminathan. 2024. On overcoming miscalibrated conversational priors in llm-based chatbots. [arXiv preprint arXiv:2406.01633](#).
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. [arXiv preprint arXiv:2410.21276](#).
- Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, and 17 others. 2022. Language Models (Mostly) Know What They Know. (arXiv:2207.05221).
- Adam Tauman Kalai, Ofir Nachum, Santosh S Vempala, and Edwin Zhang. 2025. Why language models hallucinate. [arXiv preprint arXiv:2509.04664](#).
- Sanyam Kapoor, Nate Gruver, Manley Roberts, Katherine Collins, Arka Pal, Umang Bhatt, Adrian Weller, Samuel Dooley, Micah Goldblum, and Andrew Gordon Wilson. 2024. Large Language Models Must Be Taught to Know What They Don’t Know. (arXiv:2406.08391).
- Wai-Chung Kwan, Xingshan Zeng, Yuxin Jiang, Yufei Wang, Liangyou Li, Lifeng Shang, Xin Jiang, Qun Liu, and Kam-Fai Wong. 2024. Mt-eval: A multi-turn capabilities evaluation benchmark for large language models. In [Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing](#), pages 20153–20177.
- Philippe Laban, John Canny, and Marti A Hearst. 2021. What’s the latest? a question-driven news chatbot. [arXiv preprint arXiv:2105.05392](#).
- Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. Llms get lost in multi-turn conversation. [arXiv preprint arXiv:2505.06120](#).
- Philippe Laban, Lidiya Murakhovska, Caiming Xiong, and Chien-Sheng Wu. 2023. Are you sure? challenging llms leads to performance drops in the flipflop experiment. [arXiv preprint arXiv:2311.08596](#).
- Belinda Z. Li, Been Kim, and Zi Wang. 2025. Quest-Bench: Can LLMs ask the right question to acquire information in reasoning tasks? (arXiv:2503.22674).
- Bo Li, Peng Qi, Bo Liu, Shuai Di, Jingen Liu, Jiquan Pei, Jinfeng Yi, and Bowen Zhou. 2022a. [Trustworthy ai: From principles to practices](#). [Preprint](#), arXiv:2110.01167.
- Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. 2023. Taco: Topics in algorithmic code generation dataset. [arXiv preprint arXiv:2312.14852](#).
- Ruosen Li, Ruochen Li, Barry Wang, and Xinya Du. 2024. Iqa-eval: Automatic evaluation of human-model interactive question answering. [Advances in Neural Information Processing Systems](#), 37:109894–109921.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustín Dal Lago, and 1 others. 2022b. Competition-level code generation with alphacode. [Science](#), 378(6624):1092–1097.
- Zhenwen Liang, Dian Yu, Wenhao Yu, Wenlin Yao, Zhihan Zhang, Xiangliang Zhang, and Dong Yu. 2024. Mathchat: Benchmarking mathematical reasoning and instruction following in multi-turn interactions. [arXiv preprint arXiv:2405.19444](#).
- Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Teaching Models to Express Their Uncertainty in Words. (arXiv:2205.14334).
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. [Lost in the middle: How language models use long contexts](#). [Transactions of the Association for Computational Linguistics](#), 12:157–173.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. 2025. Understanding r1-zero-like training: A critical perspective. [arXiv preprint arXiv:2503.20783](#).
- Nishanth Madhusudhan, Sathwik Tejaswi Madhusudhan, Vikas Yadav, and Masoud Hashemi. 2025. [Do LLMs Know When to NOT Answer? Investigating Abstinence Abilities of Large Language Models](#). In [Proceedings of the 31st International Conference on Computational Linguistics](#), pages 9329–9345. Association for Computational Linguistics.

- Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. (arXiv:2402.04249).
- Felix B. Mueller, Rebekka Görge, Anna K. Bernzen, Janna C. Pirk, and Maximilian Poretschkin. 2024. Llms and memorization: On quality and specificity of copyright compliance. In Proceedings of the Seventh AAAI/ACM Conference on AI, Ethics, and Society (AI/ES-24) - Full Archival Papers, October 21-23, 2024, San Jose, California, USA - Volume 1, pages 984–996. AAAI Press.
- Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafford, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, and 21 others. 2024. 2 olmo 2 furious. Preprint, arXiv:2501.00656.
- OpenAI. 2024. Gpt-4o mini: advancing cost-efficient intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>.
- OpenAI. 2025a. Introducing gpt-4.1 in the api. <https://openai.com/index/gpt-4-1/>.
- OpenAI. 2025b. OpenAI o3 and o4-mini System Card — openai.com. <https://openai.com/index/o3-o4-mini-system-card/>. [Accessed 08-05-2025].
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, and 1 others. 2022. Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35:27730–27744.
- Christian Poelitz and Nick McKenna. 2025. Synthetic clarification and correction dialogues about data-centric tasks—a teacher-student approach. arXiv preprint arXiv:2503.14167.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, and 25 others. 2025. Qwen2.5 technical report. Preprint, arXiv:2412.15115.
- Ashwin Ram, Rohit Prasad, Chandra Khatri, Anu Venkatesh, Raefer Gabriel, Qing Liu, Jeff Nunn, Behnam Hedayatnia, Ming Cheng, Ashish Nagar, and 1 others. 2018. Conversational ai: The science behind the alexa prize. arXiv preprint arXiv:1801.03604.
- Siva Reddy, Danqi Chen, and Christopher D Manning. 2019. Coqa: A conversational question answering challenge. Transactions of the Association for Computational Linguistics, 7:249–266.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, and 1 others. 2024. Deepseek-math: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300.
- Ved Sirdeshmukh, Kaustubh Deshpande, Johannes Mols, Lifeng Jin, Ed-Yeremai Cardona, Dean Lee, Jeremy Kritz, Willow Primack, Summer Yue, and Chen Xing. 2025. Multichallenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier llms. arXiv preprint arXiv:2501.17399.
- Aviv Slobodkin, Omer Goldman, Avi Caciularu, Ido Dagan, and Shauli Ravfogel. 2023. The Curious Case of Hallucinatory (Un)answerability: Finding Truths in the Hidden States of Over-Confident Large Language Models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 3607–3625. Association for Computational Linguistics.
- Petru Soviany, Radu Tudor Ionescu, Paolo Rota, and Nicu Sebe. 2022. Curriculum learning: A survey. Preprint, arXiv:2101.10382.
- Yi Su, Dian Yu, Linfeng Song, Juntao Li, Haitao Mi, Zhaopeng Tu, Min Zhang, and Dong Yu. 2025. Crossing the reward bridge: Expanding rl with verifiable rewards across diverse domains. arXiv e-prints, pages arXiv–2503.
- Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D. Manning. 2023. Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback. (arXiv:2305.14975).
- Roman Vashurin, Ekaterina Fadeeva, Artem Vazhentsev, Lyudmila Rvanova, Daniil Vasilev, Akim Tsvigun, Sergey Petrakov, Rui Xing, Abdelrahman Sadallah, Kirill Grishchenkov, Alexander Panchenko, Timothy Baldwin, Preslav Nakov, Maxim Panov, and Artem Shelmanov. 2024. Benchmarking Uncertainty Quantification Methods for Large Language Models with LM-Polygraph. Transactions of the Association for Computational Linguistics, 13:220–248.
- Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. 2024. Mint: Evaluating llms in multi-turn interaction with tools and language feedback. In The Twelfth International Conference on Learning Representations.
- Bingbing Wen, Jihan Yao, Shangbin Feng, Chenjun Xu, Yulia Tsvetkov, Bill Howe, and Lucy Lu Wang. 2025. Know your limits: A survey of abstention in large language models. Transactions of the Association for Computational Linguistics, 13:529–556.

- Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2024. Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs. (arXiv:2306.13063).
- Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. Berkeley function calling leaderboard. https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. Preprint, arXiv:2505.09388.
- Zhangyue Yin, Qiushi Sun, Qipeng Guo, Jiawen Wu, Xipeng Qiu, and Xuanjing Huang. 2023. Do Large Language Models Know What They Don't Know? In Findings of the Association for Computational Linguistics: ACL 2023, pages 8653–8665. Association for Computational Linguistics.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, and 1 others. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Tong Zhang, Peixin Qin, Yang Deng, Chen Huang, Wenqiang Lei, Junhong Liu, Dingnan Jin, Hongru Liang, and Tat-Seng Chua. 2024. CLAMBER: A Benchmark of Identifying and Clarifying Ambiguous Information Needs in Large Language Models. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 10746–10766. Association for Computational Linguistics.
- Xiaojiang Zhang, Jinghui Wang, Zifei Cheng, Wenhao Zhuang, Zheng Lin, Minglei Zhang, Shaojie Wang, Yinghan Cui, Chao Wang, Junyi Peng, and 1 others. 2025. Srpo: A cross-domain implementation of large-scale reinforcement learning on llm. arXiv preprint arXiv:2504.14286.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, and 1 others. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. Advances in Neural Information Processing Systems, 36:46595–46623.
- George Kingsley Zipf. 1949. Human behavior and the principle of least effort: An introduction to human ecology. Addison-Wesley Press.

A Discussion on Evaluation Metrics

For the Abstain Score, we use GPT-4o as an LLM judge. Importantly, the judge is not asked to assess subjective response quality; it only determines whether the response expresses abstention intent, e.g., that the question cannot be answered from the available context or that more information is needed. In this sense, LLM-as-a-judge serves as a semantic classifier for abstention behavior.

This also explains why training and evaluation use different criteria. During training, we follow standard RLVR and use deterministic string matching, treating Abstain as a verifiable terminal action. For evaluation, however, exact matching would unfairly penalize baseline models that were not trained to emit the literal token Abstain and may instead express uncertainty in natural language. Thus, LLM-as-a-judge is used only to normalize semantically equivalent abstention expressions across models.

We further validate this choice with human evaluation on 100 math examples (50 solvable and 50 unsolvable), comparing human labels with GPT-4o judgments. As shown in Table 4, the agreement is high for both the Qwen2.5-7B baseline (0.92) and our Qwen2.5-7B RLAAR model (0.97), supporting the reliability of this metric.

Model	Human-LLM Consistency
Qwen2.5-7B Baseline	0.92
Qwen2.5-7B (Ours, RLAAR)	0.97

Table 4: Agreement between human annotations and GPT-4o on whether a response expresses abstention. We randomly sample 100 math examples in total, including 50 solvable and 50 unsolvable cases.

B Examples

In this section, we provide a failure example of the baseline model and a success example of our method to show the desired behavior of the model.

Example B shows a typical failure example of most of the existing models. Every time the user provides some new information, the model tends to answer the question prematurely without checking the solvability of the question, and thus results in an incorrect answer.

On the other hand, Example B shows a success example of our method, which shows the desired behavior when facing this kind of situation. Every time the user provides some new information, the model will check the solvability of the question first. If the question is not solvable, the model will

abstain from the question. Only if the model is sure that the question is solvable, it answers the question.

Failure Example

(Round 1) User: *how many vines does Steve need to grow his cherry tomatoes?*

Assistant: To determine how many vines Steve needs to grow his cherry tomatoes, we need to know: 1. How many cherry tomato plants Steve wants to grow. 2. How many vines each cherry tomato plant requires. Step 1: Assume a standard setup - Each cherry tomato plant typically requires 1 vine to grow. - Each vine can produce approximately 3-4 cherry tomato plants. Step 2: Estimate based on the number of plants Let's assume Steve wants to grow 10 cherry tomato plants. - If each plant needs 1 vine, then Steve needs 10 vines. Step 3: Final Answer 10 So, Steve needs 10 vines to grow his cherry tomatoes.

(Premature Answer)

(Round 2) User: *He eats 6 tomatoes every day.*

Assistant: Let's determine how many tomatoes Steve eats in a week. Step 1: Calculate daily consumption - Steve eats 6 tomatoes per day. Step 2: Calculate weekly consumption - There are 7 days in a week. - So, total tomatoes eaten in a week = 6 tomatoes/day $\times$ 7 days. $6 \times 7 = 42$. Step 3: Final Answer 42 Steve eats 42 tomatoes in a week.

(Premature Answer)

(Round 3) User: *Steve wants to grow his own cherry tomatoes to eat more.*

Assistant: To determine how many vines Steve needs to grow his own cherry tomatoes, we need to make some assumptions and use logical reasoning. Step 1: Assume the number of plants Steve wants to grow Let's assume Steve wants to grow 10 cherry tomato plants. Step 2: Determine how many vines are needed per plant Each cherry tomato plant typically requires 1 vine to grow. Step 3: Total number of vines needed Number of vines = Number of plants $\times$ Vines per plant = $10 \times 1 = 10$ Step 4: Consider the tomato yield per vine Each vine can produce 3-4 cherry tomato plants, so: Tomato yield per vine = 3~4 plants If Steve wants to grow 10 plants, and each vine can yield 3 plants, then: Number of vines = $\frac{10}{3} \approx 3.33$ Since you can't grow a fraction of a vine, Steve would need 4 vines to grow 12 cherry tomato plants. Final Answer: 4 Steve needs 4 vines to grow his own cherry tomatoes.

(Premature Answer)

...

(Round Final) User: *Each vine can produce 3 tomatoes weekly.*

Assistant: To determine how many vines Steve needs to produce 30 tomatoes per week, we follow these steps: Step 1: Understand the production rate - Each vine produces 3 tomatoes per week. Step 2: Calculate the number of vines needed Number of vines = $\frac{\text{Total tomatoes needed}}{\text{Tomatoes per vine}} = \frac{30}{3} = 10$ Final Answer: 10 Steve needs 10 vines to produce 30 tomatoes per week.

(Incorrect Answer)

Success Example

(Round 1) User: *how many vines does Steve need to grow his cherry tomatoes?*

Assistant: To determine how many vines Steve needs to grow his cherry tomatoes, we would need more specific information, such as: - The number of cherry tomato plants Steve wants to grow. - The number of vines each plant requires. - The number of plants per vine (if applicable). - Any constraints or limitations (e.g., space, resources, etc.). Since none of these details are provided in the question, it is not possible to determine the exact number of vines Steve needs. Reasoning: - Without knowing how many plants Steve wants, or how many vines each plant requires, we cannot compute the number of vines. - Similarly, if there are multiple plants per vine, this would also affect the total number of vines needed. Conclusion: The question is not solvable with the information given. Abstain

(Abstain)

(Round 2) User: *He eats 6 tomatoes every day.*

Assistant: We are given the following information: - Steve eats 6 tomatoes every day. Step 1: Understand the problem We are asked to find how many vines Steve needs to grow his cherry tomatoes. However, we do not have any information about: - How many plants Steve wants to grow. - How many tomatoes each plant produces. - How many vines are needed per plant. Step 2: Identify missing information The question is not solvable unless we have the following: - The number of cherry tomato plants Steve wants to grow. - The number of tomatoes each plant produces. - The number of vines required per plant. Step 3: Determine if the question is solvable Since we do not have the above information, we cannot compute how many vines Steve needs. Conclusion: The question cannot be solved without additional information. Abstain

(Abstain)

(Round 3) User: *Steve wants to grow his own cherry tomatoes to eat more.*

Assistant: To determine how many vines Steve needs to grow his own cherry tomatoes, we must first understand the relationship between vines, tomatoes, and Steve's needs. Step 1: Understand the context Steve is planning to grow his own cherry tomatoes to supply his needs. This implies: - He wants to grow cherry tomatoes. - He does not need to buy tomatoes from a store. - He is planting and growing his own tomatoes. Step 2: Gather relevant information The key variables we need to know are: - How many tomatoes Steve needs per day: From the previous question, we know that Steve eats 6 tomatoes every day. - How many tomatoes each vine produces: This information is not provided in the question. - How many vines are needed per plant: This information is not provided in the question. Step 3: Evaluate the problem Without knowing: - How many tomatoes Steve needs in total (i.e., total daily or monthly consumption), - How many tomatoes each vine produces, - How many vines are required per plant, we cannot compute the exact number of vines Steve needs. Step 4: Determine if the problem is solvable The question is not solvable due to a lack of necessary data. It is a mathematical problem that requires unknown parameters to compute the answer. Final Conclusion: The question cannot be solved with the information provided. Abstain

(Abstain)

...

(Round Final) User: *Each vine can produce 3 tomatoes weekly.*

Assistant: To determine how many vines Steve needs to grow his cherry tomatoes, we now use the following known information: Step 1: Known Information - Steve eats 6 tomatoes every day. - Steve eats twice as many tomatoes as his girlfriend, so: Girlfriend's daily consumption = $\frac{6}{2} = 3$ tomatoes/day Total daily consumption = $6 + 3 = 9$ tomatoes/day - Each vine can produce 3 tomatoes every week. Step 2: Convert Weekly Yield to Daily Yield Since each vine produces 3 tomatoes per week, we calculate the daily yield per vine: Daily yield per vine = $\frac{3 \text{ tomatoes/week}}{7 \text{ days/week}} = \frac{3}{7}$ tomatoes/day Step 3: Calculate Total Daily Tomatoes Needed We have already determined: Total daily tomatoes needed = 9 tomatoes/day Step 4: Calculate Total Number of Vines Needed Now, we calculate the number of vines required to produce the 9 tomatoes per day. Number of vines needed = $\frac{9}{\frac{3}{7}} = 9 \times \frac{7}{3} = 21$ vines

Final Answer: 21

(Correct Answer)

C Pseudo Code

The pseudo code of the RLAAR method is shown in Algorithm 1 and Algorithm 2.

Algorithm 1: Curriculum RL with Verifiable Accuracy and Abstention Rewards (RLAAR)

Input: Policy π_θ ; Dataset $\mathcal{D}$; Max shards $K_{\max}$; Abstention ratio m ; Threshold ratio ρ ; Window size w ; Batch size B

Output: Trained policy π_θ

1 Stage 1: Threshold Establishment;

```
2  $K \leftarrow 1$ ;  
3 for  $i = 1$  to  $w$  do  
4   trajectories  $\leftarrow []$ , rewards  $\leftarrow []$ ;  
5   for  $j = 1$  to  $B$  do  
6     task  $\sim \text{Sample}(\mathcal{D})$ ;  
7      $\tau \leftarrow \text{SolvableSingleRollout}(\pi_\theta, \text{task}, K=1)$ ;  
8      $r \leftarrow r_{\text{acc}}(\tau)$ ;  
9     trajectories.append( $\tau$ ); rewards.append( $r$ );  
10   $\pi_\theta \leftarrow \text{UpdatePolicy}(\pi_\theta, \text{trajectories}, \text{rewards})$ ;  
11  $\bar{r}_0 \leftarrow \text{MovingAverage}(\text{all\_rewards}, \text{window} = w)$ , threshold  $\leftarrow \rho \times \bar{r}_0$ ;
```

12 Stage 2: Main Training (Curriculum);

```
13  $K \leftarrow 2$ ;  
14 while  $K \leq K_{\max}$  do  
15   trajectories  $\leftarrow []$ , rewards  $\leftarrow []$ ;  
16   for  $i = 1$  to  $B$  do  
17     task  $\sim \text{Sample}(\mathcal{D})$ ;  
18     if random()  $< m$  then  
19        $\tau \leftarrow \text{UnsolvableMultiRollout}(\pi_\theta, \text{task}, M)$ ;  
20        $r \leftarrow r_{\text{abs}}(\tau)$ ;  
21     else  
22        $\tau \leftarrow \text{SolvableMultiRollout}(\pi_\theta, \text{task}, K)$ ;  
23        $r \leftarrow r_{\text{acc}}(\tau)$ ;  
24     trajectories.append( $\tau$ ); rewards.append( $r$ );  
25    $\pi_\theta \leftarrow \text{UpdatePolicy}(\pi_\theta, \text{trajectories}, \text{rewards})$ ,  $\bar{r} \leftarrow \text{MovingAverage}(\text{rewards}, \text{window} = w)$ ;  
26   if  $\bar{r} \geq \text{threshold}$  then  
27      $K \leftarrow K + 1$ ;
```

28 Stage 3: Randomized Training;

```
29 for  $i = 1$  to Max_steps do  
30    $K \leftarrow \text{random\_int}(1, K_{\max})$ , trajectories  $\leftarrow []$ , rewards  $\leftarrow []$ ;  
31   for  $i = 1$  to  $B$  do  
32     task  $\sim \text{Sample}(\mathcal{D})$ ;  
33     if random()  $< m$  then  
34        $\tau \leftarrow \text{UnsolvableMultiRollout}(\pi_\theta, \text{task}, M)$ ;  
35        $r \leftarrow r_{\text{abs}}(\tau)$ ;  
36     else  
37        $\tau \leftarrow \text{SolvableMultiRollout}(\pi_\theta, \text{task}, K)$ ;  
38        $r \leftarrow r_{\text{acc}}(\tau)$ ;  
39     trajectories.append( $\tau$ ); rewards.append( $r$ );  
40    $\pi_\theta \leftarrow \text{UpdatePolicy}(\pi_\theta, \text{trajectories}, \text{rewards})$ ;  
41 return  $\pi_\theta$ ;
```

Algorithm 2: Multi-turn Rollout Functions

```
1 Function SolvableSingleRollout( $\pi_\theta$ , task  $T$ ,  $K=1$ ):  
2    $s_1 \leftarrow \text{FULLQUESTION}(T)$   
3    $a_1 \sim \pi_\theta(\cdot \mid s_1)$   
4    $\tau \leftarrow [(s_1, a_1)]$   
5   return  $\tau$   
  
6 Function SolvableMultiRollout( $\pi_\theta$ , task  $T$ ,  $K$ ):  
7    $S \leftarrow \{s_1, s_2, \dots, s_K\} \leftarrow \text{SHARDQUESTION}(T, K)$   
8    $\tau \leftarrow []$ ;  $c_1 \leftarrow s_1$   
9   for  $k \leftarrow 1$  to  $K$  do  
10     $a_k \sim \pi_\theta(\cdot \mid c_k)$   
11     $\tau.\text{append}((c_k, a_k))$   
12    if  $k < K$  then  
13       $c_{k+1} \leftarrow (s_1, a_1, \dots, s_k, a_k, s_{k+1})$   
14  return  $\tau$   
  
15 Function UnsolvableMultiRollout( $\pi_\theta$ , task  $T$ ,  $M$ ):  
16   $S \leftarrow \{s_1, s_2, \dots, s_M\} \leftarrow \text{SHARDQUESTION}(T, M, \text{incomplete}=\text{True})$   
17   $\tau \leftarrow []$ ;  $c_1 \leftarrow s_1$   
18  for  $k \leftarrow 1$  to  $M$  do  
19     $a_k \sim \pi_\theta(\cdot \mid c_k)$   
20     $\tau.\text{append}((c_k, a_k))$   
21    if  $k < M$  then  
22       $c_{k+1} \leftarrow (s_1, a_1, \dots, s_k, a_k, s_{k+1})$   
23  return  $\tau$ 
```
